

FaultForge

Break your service's dependencies on purpose,
then get a repair that's proven against the same failures.

NVIDIA Nemotron 3 Ultra + Nano | Nebius Token Factory | Token Factory Sandboxes | Tavily

Nebius x NVIDIA Global AI Hackathon · Coding & Agentic Engineering track

THE PROBLEM

Outages come from the code nobody's tests run.

- A payment provider rate-limits you, and a 429 is read as "paid".
- A fulfillment API does the work, then the response times out, and the retry ships it twice.
- A webhook is delivered twice and processed twice.
- These paths almost never run in tests: the fake always answers instantly and correctly.

TODAY

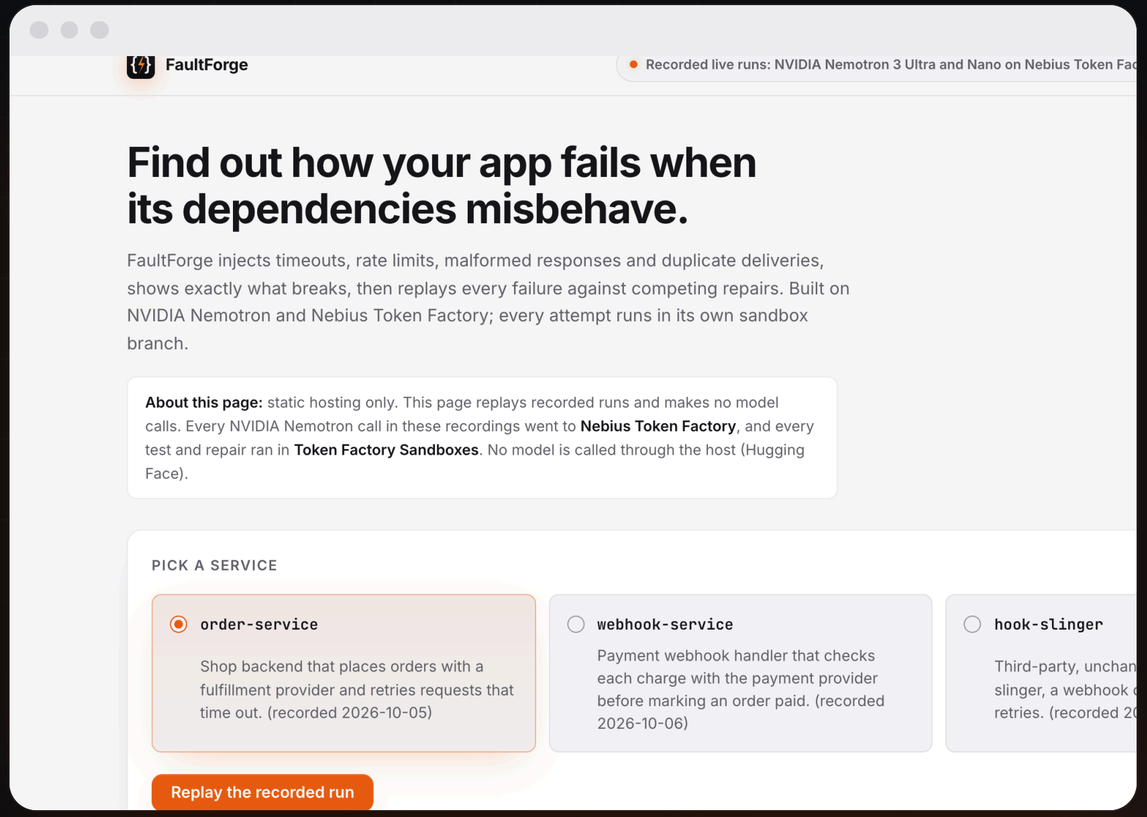
Chaos tools can **inject** a failure. A small team still has to work out what broke, write the fix, and prove the fix doesn't break something else.

FaultForge does the whole loop.

THE PRODUCT

Point it at a service. Get back a proven repair.

- **Finds** the outbound HTTP dependency and the operation at risk.
- **Breaks** it on purpose: timeouts, rate limits, malformed responses, duplicate delivery, as executable tests.
- **Repairs** each reproduced failure with competing fixes.
- **Proves** the winner: every failure replayed against every repair, in Sandboxes.
- **Ships** a pull request with the patch, the regression test and the evidence.



FLAGSHIP FINDING, FROM A LIVE RUN

One order. Two shipments.

```
01  POST /fulfillments → accepted
02  response timed out
03  retry → POST /fulfillments
04  second shipment created
```

Test written by Nemotron: exactly one fulfillment must exist.
Baseline: 2 == 1, failed.

WINNING REPAIR · 12 LINES

```
headers["Idempotency-Key"] = order_id
```

The provider can now deduplicate the retry. Replayed against every scenario: the timeout failure is **fixed**, nothing regressed, existing tests stay green. A competing repair that only cached locally was **rejected**: it fixed nothing.

HOW IT WORKS

An agent loop, with a verifier at the end.

01
Find the dependency
Static scan + Nemotron 3 Ultra

02
Research how it fails
Tavily: retries, Retry-After, idempotency keys

03
Write failure tests
Nemotron 3 Nano, faults injected in the HTTP client

04
Triage every failure
Ultra: a real bug, or a mistake in the test?

05
Plan and write repairs
Ultra plans distinct fixes, Nano writes them

06
Replay everything
Every failure × every repair, in Sandboxes

07
Revise or pick a winner
A failed repair gets one revision with the exact error

08
Open a pull request
Patch, regression test, evidence report

ARCHITECTURE

Every model call on Token Factory. Every line of model code in a sandbox.

Your service
Python, HTTP dependency

Web UI
React · live attempt tree

API
FastAPI · server-sent events

→

FAULTFORGE PIPELINE

Nemotron 3 Ultra
plan · triage · repair strategy · revision (us-central1)

Nemotron 3 Nano
failure tests · patches, thinking off (eu-north1)

Tavily
how the real dependency fails, cited

→

TOKEN FACTORY SANDBOXES

checkpoint

baseline | **scenario × N**

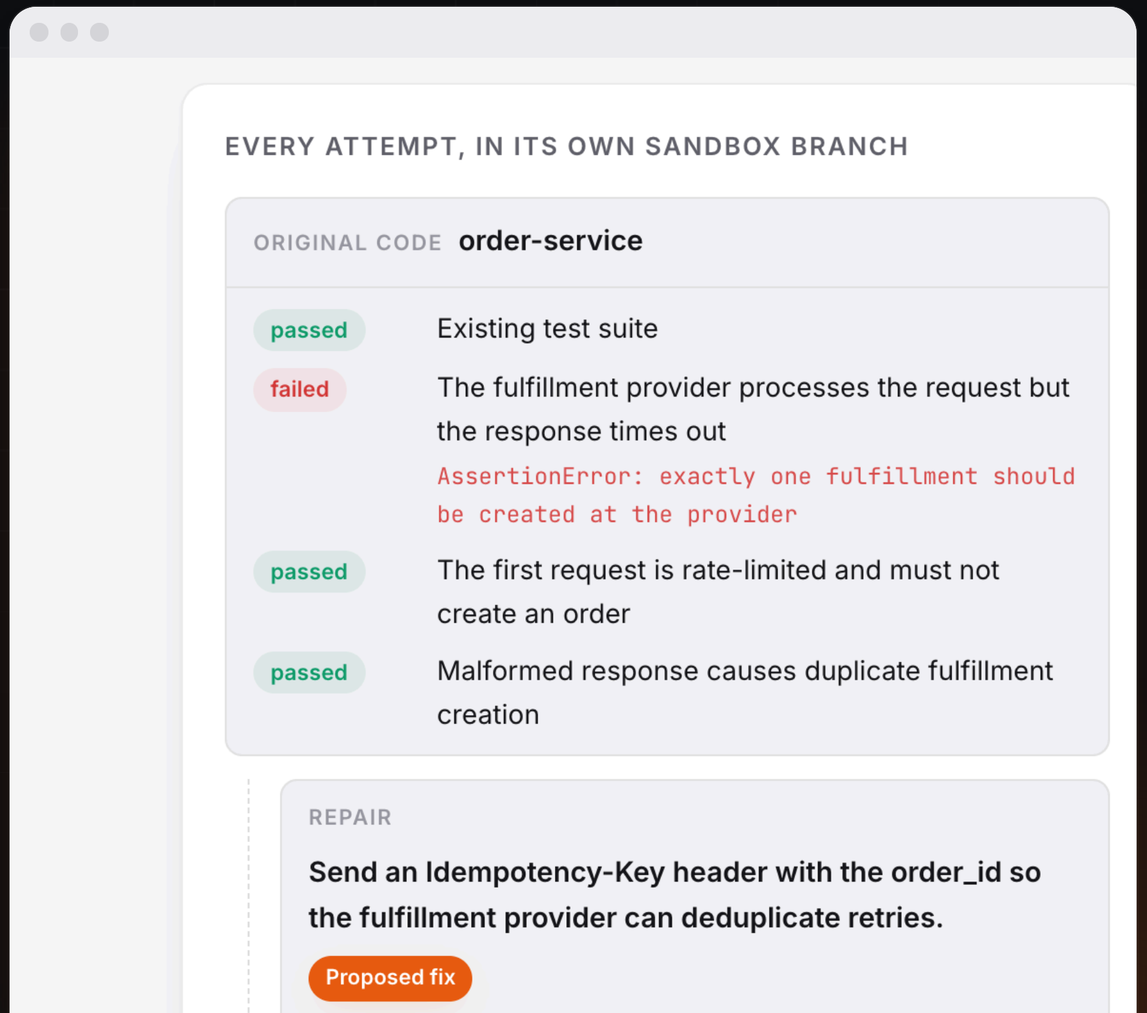
repair A | **repair B**

Each run yields a new image: every branch starts from the same checkpoint.

THE PROOF

Every failure, replayed against every repair.

- A repair wins only if it fixes a reproduced failure, breaks no existing test and regresses no scenario.
- Each attempt runs in its own sandbox branch, side by side.
- The UI shows why each attempt was accepted or rejected, with the real test output.
- Generated tests are triaged too: a broken test goes back to the writer instead of "fixing" the app.



RESULTS

Three real failures. Three verified fixes.

`order-service`

Retry after a timeout ships the order twice

Fix: send an Idempotency-Key with the order id.

`webhook-service`

A rate-limited check marks an unpaid order paid

HTTP 429 from the payment provider was read as success. Fix: check the response status first.

`hook-slinger · third party`

A duplicate delivery is processed twice

An unchanged open-source project (rednafi/hook-slinger, MIT). Fix: a stable idempotency key through the retry queue.

All three repaired live on Nemotron + Token Factory Sandboxes; every winning patch was read, not just its verdict.

NUMBERS

Small-team reliability, for cents.

6/8

live runs ended in a genuine, verified repair after triage

$0.08

per full run: discover, research, test, triage, repair, verify

2–3min

from a fresh start to a verified repair on Sandboxes

180+

tests in the FaultForge codebase, built test-first

Hardening: a code review (15 issues, all fixed), a two-model security review of the public surface, and a daily cap on live runs so a public demo can't drain credit.

WHAT WE LEARNED

Make the small model write. Make the big model judge.

Code inside JSON fails at every size

Fenced code blocks with a worked example fixed it. Bigger writers did no better than Nano once output was validated.

Read the patches, not the verdicts

Early "repairs" made the app tolerate a broken test. Ultra's triage step stopped that: 6 of 8 genuine.

Turn Nano's thinking off for writing

Runs went from about 15 minutes to about 1. Ultra keeps reasoning for planning and review.

Branching sandboxes fit agents

One checkpoint, many branches: comparing alternatives becomes the natural shape of the work.

WHAT'S NEXT

From a hackathon loop to a pull-request check.

More faults

Partial responses, slow streams, connection resets; dependencies beyond HTTP: queues and databases.

A GitHub App

Run FaultForge on every pull request that touches dependency code, and comment with the evidence.

Safer defaults

Idempotency keys unique per destination by default, a caveat found in the hook-slinger run.

Try FaultForge

Demo	realramy-faultforge.static.hf.space
Video	youtu.be/zSOExa4RrCc
Code	github.com/Ramy888/NebiusxNvidia-FaultForge

NVIDIA Nemotron 3 Ultra + Nano · Nebius Token Factory · Token Factory Sandboxes · Tavily